ML Systems 101: From Jupyter to Production

Key Takeaways

Production ML is a system, not a model

The model is typically 5–10% of production code. The rest is data pipelines, feature engineering, serving infrastructure, monitoring, and glue. A brilliant model inside a broken system fails silently.

Training-serving skew is the #1 silent killer

When the features at serving time differ from training time, accuracy drops without a single error log. You must log and monitor feature distributions at serving time, not just training time.

Technical debt compounds fast in ML

Undeclared consumers, pipeline jungles, and correction cascades (Sculley et al.) are ML-specific forms of debt. They are invisible until they are catastrophic. Address them at design time.

Answer the 5 questions before writing any code

Problem → Data → Model → Serving → Monitoring. Skipping any one of them guarantees a rebuild later.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

Q1. A model trained on last month's data is deployed. Three months later, accuracy has dropped 15% but no errors are logged. What failure mode is this, and how would you detect it before users notice?

Q2. Your team has 5 scripts that all read from the same feature table, but only 2 of them update it and nobody documented which. What ML technical debt pattern does Sculley et al. call this?

Q3. You are designing a fraud detection system. Apply the 5-question framework — write one concrete sentence answering each of the 5 questions for this system.

Q4. What is the difference between a model registry and just storing model files on S3? Name two things a registry gives you that a file store does not.

Coding / Design Exercises

Work through these independently or in pairs. No starter code — derive the approach from the lecture.

Exercise 1: Audit a Notebook

Take any ML notebook you wrote in a previous course (CV, NLP, or Applied AI). Identify which of the six production components are missing. Write a one-paragraph "production gap analysis": what would need to be built to actually deploy this model at scale?

Exercise 2: The 5-Question Design Doc

Pick one of these systems: (a) Spotify song recommendation, (b) Gmail spam filter, (c) Uber surge pricing model. Apply the 5-question framework. Produce a one-page design doc with all five sections answered concretely. Focus on data and serving — those are where most designs go wrong.

Exercise 3: Find Your Own Technical Debt

Read Section 2 of Sculley et al. 2015 (search: "Hidden Technical Debt in ML Systems NeurIPS 2015"). Identify two debt patterns you have personally created in past projects. Write one sentence each: what the pattern was, and how you would fix it if you had to deploy that code today.

Challenge Problem

The ML Postmortem

Pick any public ML failure: Amazon's hiring algorithm (2018), Zillow's iBuying collapse (2021), or Uber's surge pricing PR crisis. All are well-documented online. Write a structured postmortem: (1) What went wrong — in ML system terms, not business terms. (2) Which component(s) failed. (3) What monitoring would have caught it earlier. (4) What you would design differently. One paragraph per section.

Further Reading

- **Sculley et al. (2015) — Hidden Technical Debt in ML Systems** — NeurIPS 2015, freely available via Google Scholar
  The paper we discussed. Read at least Sections 1–3. The rest is worth your time once you have built something real.
- **Chip Huyen — Designing Machine Learning Systems, Ch. 1–2** — O'Reilly, 2022 (course primary text)
  Chapter 1 covers the ML system landscape. Chapter 2 covers project scoping. Directly relevant to today.
- **Andrej Karpathy — Software 2.0** — medium.com/@karpathy, 2017
  A 10-minute read on why ML is a new programming paradigm. Sets the context for why production ML is hard.
- **Eugene Yan — Patterns for Building LLM-based Systems** — eugeneyan.com, 2023
  Free blog post series. Directly relevant from Week 4 onwards. Start with the RAG and evaluation posts.

Online Resources

- **MLflow Docs** — mlflow.org — experiment tracking and model registry (we use this from Week 3)
- **Evidently AI Blog** — evidentlyai.com — ML monitoring and data drift in plain English
- **Chip Huyen's Blog** — huyenchip.com — real-world ML system war stories
- **Google's ML Crash Course** — ml-crash-course/mlsystems — systems chapter is free
- **DAIR.AI ML News** — weekly newsletter on production ML
- **The Batch (DeepLearning.AI)** — practical AI news with system design examples

Answers

A1. Data drift (the world changed, the model did not) combined with silent degradation (no error logs because the model is still returning valid predictions — they are just wrong). Detect it by monitoring feature distributions at serving time with tools like Evidently, and tracking business metrics (conversion rate, churn) alongside model metrics.

A2. Undeclared consumers — a pattern where multiple downstream consumers depend on a data source that has no single owner and no documented contract. When one team changes the table schema, the other 3 scripts break silently.

A3. Accept any reasonable answers that cover: Problem (detect fraudulent transactions before they complete), Data (transaction events with user history, merchant data, device fingerprint), Model (binary classifier with $<$100ms latency requirement), Serving (real-time synchronous, embedded in payment flow), Monitoring (track false positive rate alongside fraud catch rate — both matter for the business).

A4. A model registry gives you: (1) versioning with metadata (who trained it, on what data, with what metrics) — S3 gives you a filename; (2) lifecycle management (staging / production / archived states with promotion workflows) — S3 has no concept of production readiness; also: audit trail, rollback to previous version with one command.